

Aubrée Romain
10/05/2026
SLAM

Beauvallet
Semestre 2

Groupe 1

Bloc 3 TD 9

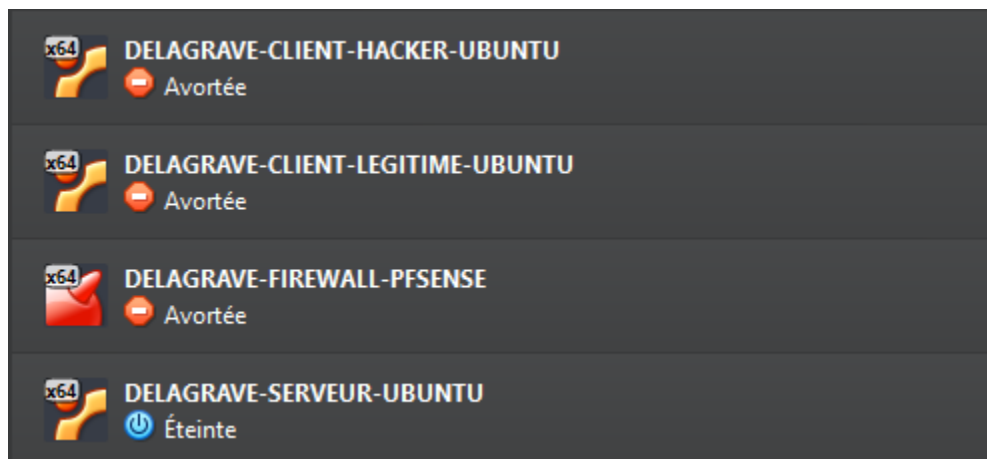
Contexte	2
Installation des VM	2
Injection SQL	5
Contre-mesure de codage sécurisé	7
Conclusion	9

Contexte

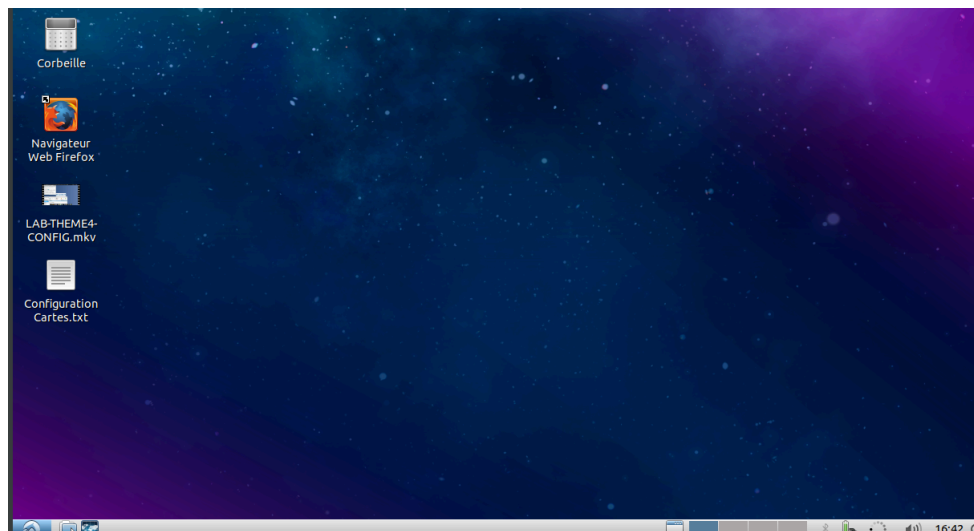
Ce TP va nous faire découvrir comment on pirate et comment on protège un site web. On va utiliser un réseau avec plusieurs machines virtuelles pour jouer deux rôles : d'abord le hacker qui veut voler la liste des membres, et ensuite le développeur qui doit corriger les failles pour sécuriser le site.

Installation des VM

Nous utiliserons **Oracle VirtualBox** :



On lance la **VM “DELAGRAVE-CLIENT-LEGITIME-UBUNTU”** :



Méthode en mode **“Manuel”** avec les paramètres suivant (comme montré dans la vidéo) :

Modification de Connexion filaire 4

Nom de la connexion : Connexion filaire 4

Général Ethernet Sécurité 802.1X DCB Proxy Paramètres IPv4 Paramètres IPv6

Méthode : Manuel

Adresses

Adresse	Masque de réseau	Passerelle
192.168.50.10	24	192.168.50.254

Ajouter Supprimer

Serveurs DNS : 8.8.8.8

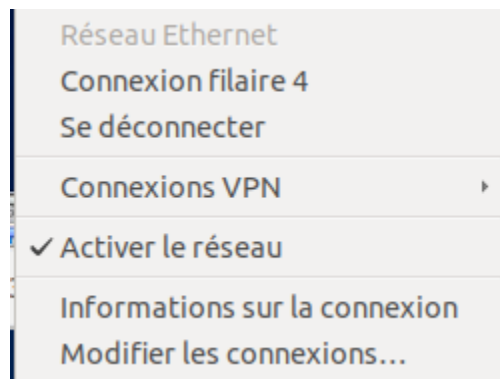
Domaines de recherche :

ID de client DHCP :

☐ Requiert un adressage IPv4 pour que cette connexion fonctionne

Routes...

On doit désactiver et réactiver la carte réseau ensuite :

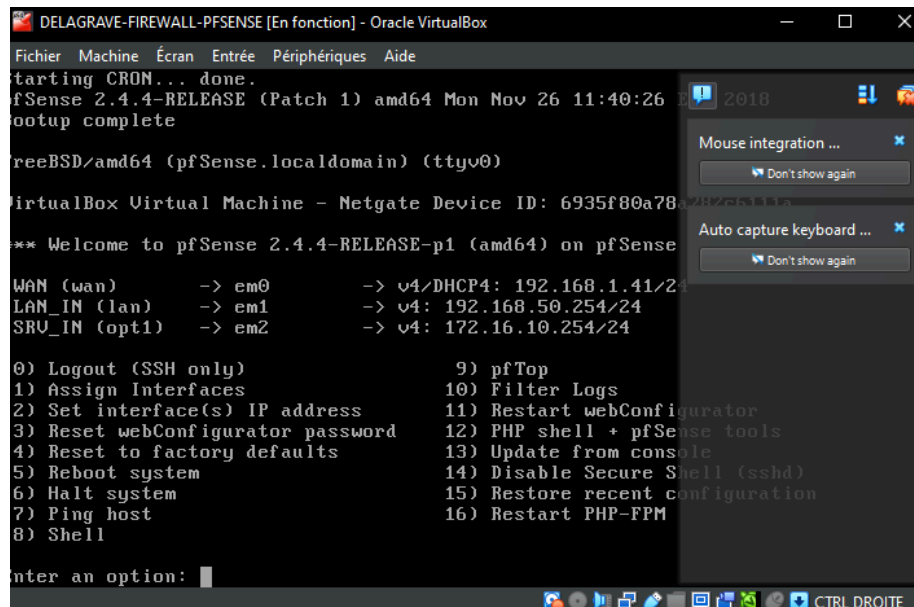


on utilise ensuite **“ip a s”** pour vérifier la configuration :

```
link/ether 08:00:27:d8:63:73 brd ff:ff:ff:ff:ff:ff
inet 192.168.50.10/24 brd 192.168.50.255 scope global noprefixroute enp0s3
    valid_lft forever preferred_lft forever
inet6 fe80::af3a:837c:9a40:62fa/64 scope link noprefixroute
    valid_lft forever preferred_lft forever
```

Après avoir démarré le serveur, et le pare-feu, le lien de la vidéo “**172.16.10.**

5/mutillidae/” fonctionne :



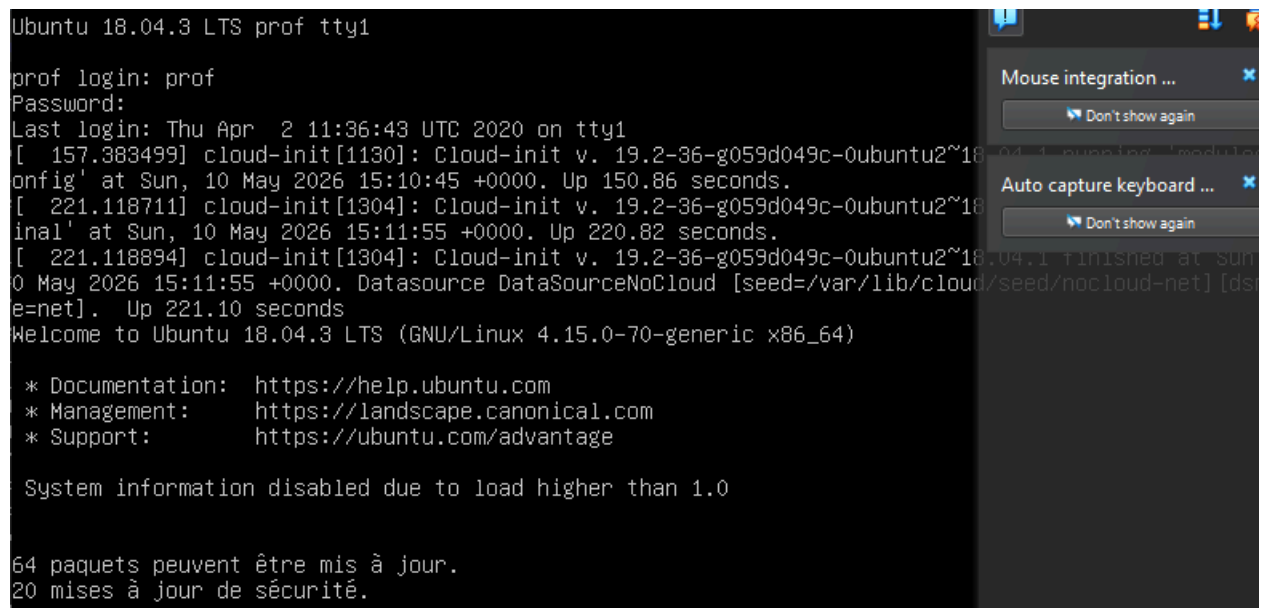
```
DELGRAVE-FIREWALL-PFSENSE [En fonction] - Oracle VirtualBox
Fichier Machine Écran Entrée Périphériques Aide
starting CRON... done.
pfSense 2.4.4-RELEASE (Patch 1) amd64 Mon Nov 26 11:40:26 2018
bootup complete

FreeBSD/amd64 (pfSense.localdomain) (ttyv0)
VirtualBox Virtual Machine - Netgate Device ID: 6935f80a78...
** Welcome to pfSense 2.4.4-RELEASE-p1 (amd64) on pfSense

WAN (wan)      -> em0      -> v4/DHCP4: 192.168.1.41/24
LAN_IN (lan)   -> em1      -> v4: 192.168.50.254/24
SRV_IN (opt1)  -> em2      -> v4: 172.16.10.254/24

0) Logout (SSH only)          9) pfTop
1) Assign Interfaces          10) Filter Logs
2) Set interface(s) IP address 11) Restart webConfigurator
3) Reset webConfigurator password 12) PHP shell + pfSense tools
4) Reset to factory defaults    13) Update from console
5) Reboot system              14) Disable Secure Shell (sshd)
6) Halt system                15) Restore recent configuration
7) Ping host                  16) Restart PHP-FPM
8) Shell

Enter an option: 
```



```
Ubuntu 18.04.3 LTS prof tty1
prof login: prof
Password:
Last login: Thu Apr  2 11:36:43 UTC 2020 on tty1
[ 157.383499] cloud-init[1130]: Cloud-init v. 19.2-36-g059d049c-0ubuntu2~18.04.1 finished at Sun
onfig' at Sun, 10 May 2026 15:10:45 +0000. Up 150.86 seconds.
[ 221.118711] cloud-init[1304]: Cloud-init v. 19.2-36-g059d049c-0ubuntu2~18.04.1 finished at Sun
inal' at Sun, 10 May 2026 15:11:55 +0000. Up 220.82 seconds.
[ 221.118894] cloud-init[1304]: Cloud-init v. 19.2-36-g059d049c-0ubuntu2~18.04.1 finished at Sun
0 May 2026 15:11:55 +0000. Datasource DataSourceNoCloud [seed=/var/lib/cloud/seed/nocloud-net] [ds
e=net]. Up 221.10 seconds
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-70-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

System information disabled due to load higher than 1.0

64 paquets peuvent être mis à jour.
20 mises à jour de sécurité.
```

Injection SQL

On démarre la VM du Hacker et on retourne sur le site “**172.16.10.5/mutillidae/**”

Configuration de la carte réseau :

Adresses		
Adresse	Masque de réseau	Passerelle
192.168.50.20	24	192.168.50.254

Une fois sur le site, on doit aller dans "**OWASP 2017 -> Injection SQL -> SQLi Extract Data -> User Info (SQL)**"

On inscrit "'or 'a' = 'a'" dans le mot de passe :

Name

Password

[View Account Details](#)

Résultat :

```
Results for "harry".23 records found.
Username=admin
Password=adminpass
Signature=g0t r00t?

Username=adrian
Password=somepassword
Signature=Zombie Films Rock!

Username=john
Password=monkey
Signature=I like the smell of confunk

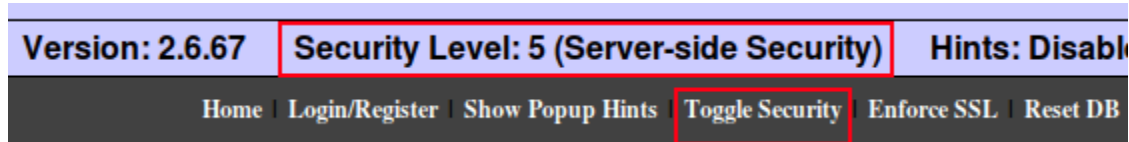
Username=jeremy
Password=password
Signature=d1373 1337 speak

Username=bryce
Password=password
```

La vulnérabilité provient d'une absence de sanitisation des inputs, permettant au payload d'altérer la structure logique de la requête **SQL** backend. Le **SGBD** exécute alors cette commande modifiée, provoquant l'exfiltration de données non autorisées.

Contre-mesure de codage sécurisé

Le niveau de sécurité était de 0 depuis le début, on va le mettre a 5 :

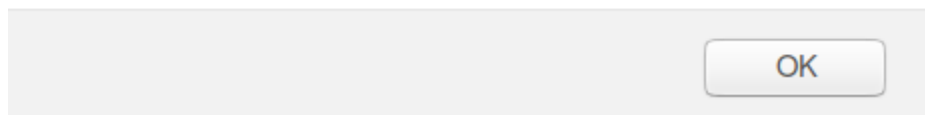


Cette fois ci, l'injection n'a pas fonctionner :

Dangerous characters detected. We can't allow these. This all powerful blacklist will stop such attempts.

Much like padlocks, filtering cannot be defeated.

Blacklisting is l33t like l33tspeak.



Sur la VM Serveur, on exécute ***"less /var/www/html/mutillidae/user-info.php"*** pour afficher la page.

Voici le code de la page **.php** qui gère ceci :

```
<?php
try{
    switch ($_SESSION["security-level"]){
        case "0": // This code is insecure
            $lEnableHTMLControls = FALSE;
            $lFormMethod = "GET";
            $lEnableJavaScriptValidation = FALSE;
            $lProtectAgainstMethodTampering = FALSE;
            $lEncodeOutput = FALSE;
            break;

        case "1": // This code is insecure
            $lEnableHTMLControls = TRUE;
            $lFormMethod = "GET";
            $lEnableJavaScriptValidation = TRUE;
            $lProtectAgainstMethodTampering = FALSE;
            $lEncodeOutput = FALSE;
            break;

        case "2":
        case "3":
        case "4":
        case "5": // This code is fairly secure
            $lEnableHTMLControls = TRUE;
            $lFormMethod = "POST";
            $lEnableJavaScriptValidation = TRUE;
            $lProtectAgainstMethodTampering = TRUE;
            $lEncodeOutput = TRUE;
            break;
    }
}
//end switch
```

```

        $lFormSubmitted = FALSE;
        if (isset($_POST["user-info-php-submit-button"]) || isset($_POST["submit-button"])) {
            $lFormSubmitted = TRUE;
        } // end if

        if ($lFormSubmitted){
            if ($lProtectAgainstMethodTampering) {
                $lUserInfoSubmitButton = $_POST["user-info-php-submit-button"];
                $lUsername = $_POST["username"];
                $lPassword = $_POST["password"];
            }else{
                $lUserInfoSubmitButton = $_REQUEST["user-info-php-submit-button"];
                $lUsername = $_REQUEST["username"];
                $lPassword = $_REQUEST["password"];
            } // end if $lProtectAgainstMethodTampering
        } // end if $lFormSubmitted

    } catch (Exception $e) {
        echo $CustomErrorHandler->FormatError($e, $lQueryString);
    } // end try;
?>

```

```

<script type="text/javascript">
<?php
if($lEnableJavaScriptValidation){
    echo "var lValidateInput = \"TRUE\" . PHP_EOL;
}else{
    echo "var lValidateInput = \"FALSE\" . PHP_EOL;
} // end if
?>

function onSubmitOfForm(/*HTMLFormElement*/ theForm){
    try{
        var lUnsafeCharacters = /[~!@#%&*()-_+=\[\]{}\\|';",./<>?]/;

        if(lValidateInput == "TRUE"){
            if (theForm.username.value.length > 15 ||
                theForm.password.value.length > 15){
                alert('Username too long. We dont want to allow too
many characters.\n\nSomeone might have enough room to enter a hack attempt.');
```


- Ce qui bloque l'attaque dans le code (Niveau 5) :
 - Le formulaire bloque toute saisie dépassant 15 caractères
 - Le code interdit l'utilisation de caractères spéciaux dangereux (comme les guillemets ' ou les tirets -)
- La bonne pratique de sécurité à en déduire :
 - Ne jamais faire confiance aux saisies de l'utilisateur. Il faut systématiquement valider la taille et nettoyer les caractères de chaque champ de formulaire avant de laisser la donnée toucher la base de données.

Conclusion

Ce TP nous a appris qu'un site mal codé est super facile à pirater avec une simple injection **SQL**. On a vu que pour bloquer ça, il ne faut jamais faire confiance à ce que l'utilisateur tape dans les formulaires. La solution, c'est de limiter le nombre de caractères et de filtrer les symboles bizarres (comme les guillemets) avant que ça arrive à la base de données.